Batches and Errors

Data Contract Validation Platform · Study Guide

What a batch is, how it is processed, and every error the platform can report: from a refused upload, to a failed run, to schema changes, down to a single wrong value in one row.

Section	Topic
1	What is a batch?
2	The four levels of errors (overview)
3	Level 1: Upload refused
4	Level 2: Run FAILED
5	Level 3: Schema changes (breaking / non-breaking)
6	Level 4: Record errors (13 error codes)
7	Contract-version changes
8	Where to see errors in the dashboard
9	Quick summary and tips

1. What is a batch?

A **batch** is **one file of records sent by a producer at one time**.

Example: every night the order service sends `orders_2024_03_01.csv` with that day's orders. That one file is one batch. When it is uploaded, the platform creates one **pipeline run** to process it.

```
1 uploaded file  =  1 batch  →  1 pipeline run  →  Input = number of rows in that file
```

It is called a *batch* because data arrives in groups (files), not one record at a time. The platform checks each batch at two levels:

- **As a whole:** is the file's structure (its columns and types) what the contract promises?
- **Record by record:** is every single row correct?

What is recorded for every batch

Captured	Meaning
Run ID	Unique ID of the processing run, e.g. RUN-20261110-061516-2b6e45
Source file name	Name of the uploaded file
Checksum (SHA-256)	Fingerprint of the file content; identical files have identical checksums
Source system	Which producer sent it (e.g. order-service)
Contract and version	Which contract the batch was checked against
Ingestion time	When it was received, started and finished
Counts	Input, valid, rejected, published, unchanged, stale skipped, late arriving
Status and failure reason	Outcome of the run and, if it failed, why

Batch numbers explained

```
Input  =  Valid  +  Rejected
```

Number	Meaning
Input	Rows in the file (the header row is not counted). Every row is read; there is no row limit, only a 50 MB file-size limit.
Valid	Rows that passed every rule
Rejected	Rows that failed at least one rule; stored in quarantine with reasons
Published	Valid rows actually written or updated in the target table
Unchanged	Valid rows already published with exactly the same data, so nothing was written
Stale skipped	Older versions of records that must not overwrite newer published data
Late arriving	Records whose event time is older than data already published

Batch outcomes (run status)

Status	Meaning
COMPLETED	Every row valid and published

Status	Meaning
COMPLETED_WITH_ERRORS	Valid rows published, invalid rows quarantined
FAILED	The whole batch was stopped; nothing published
SKIPPED_DUPLICATE	The exact same file was already processed for this contract version
PENDING / RUNNING	Waiting / being processed

Example batches from the demo

File	Input	Valid	Rejected	Published	Why
`customers.csv`	58	50	8	50	8 bad customers quarantined
`orders.csv`	100	90	10	90	10 bad orders quarantined
`orders.json`	50	50	0	0	Already published, so 'unchanged'
`orders_missing_column.csv`	20	0	0	0	Breaking change: rows not checked one by one
`orders.csv (again)`	0	0	0	0	SKIPPED_DUPLICATE: file not even read

2. The four levels of errors

Errors are grouped from the biggest (the whole file is refused) to the smallest (one value in one row).

Level	What fails	What happens to the data	Where you see it
1. Upload refused	The file	Nothing stored; error shown immediately	Data Ingestion page (red message)
2. Run FAILED	The whole batch	Nothing published; reason saved on the run	Pipeline Runs, Validation Results
3. Schema change	The file's structure	BREAKING blocks the batch; NON-BREAKING continues	Validation Results → Schema validation report
4. Record error	Single rows	Bad rows quarantined with reasons; good rows published	Rejected Records, Validation Results

```
Upload → [Level 1 checks] → Run created → Parse [Level 2] → Schema check [Level 3]
      → Record checks [Level 4] → Quarantine bad rows + Publish good rows → Run finished
```

3. Level 1: Upload refused

The file is rejected before any processing. No pipeline run is created.

HTTP error	When	Example
400 Unsupported file type	The file is not .csv or .json	data.xlsx, report.pdf
400 Empty file	The file is 0 bytes	An empty file
413 File too large	Over MAX_UPLOAD_MB (50 MB)	A very large export
404 Contract not found	The selected contract or version does not exist	Wrong contract ID / version
422 Request validation failed	A required form field is missing	No contract selected

4. Level 2: Run FAILED

The file was accepted and a run was created, but the **whole batch is stopped**. Nothing is published. The reason is saved on the run, so it appears in the metrics and on the Pipeline Runs page.

Reason	Example	Message you see
Not valid UTF-8 text	A binary file renamed to .csv	Input rejected: File is not valid UTF-8 text
Malformed CSV	A quote that is never closed: "Chennai,IN	Input rejected: Malformed CSV: ...
Malformed JSON	Missing brackets: [{"order_id": "ORD-1",	Input rejected: Malformed JSON at line ...
Empty batch	Only a header row, or []	Input rejected: ... (empty batch)
Bad header	Duplicate or empty column names	CSV header contains duplicate column names
Wrong JSON shape	Not an array of objects	JSON must be an array of objects ...
Breaking schema change	A required column is missing (see Level 3)	Blocked by 1 breaking schema change(s): MISSING_REQUIRED_COLUMN(currency). No records were published.

Reason	Example	Message you see
Unexpected processing error	Database connection lost while publishing	Unexpected processing error (error_id=...). All changes rolled back ... can be retried safely.

Important: a FAILED run never publishes anything, not even part of the batch. That is why **Retry** is always safe.

5. Level 3: Schema changes

A **schema** is the structure of the data: which columns exist and what type they are. A schema change means the file's structure differs from the contract.

- **BREAKING**: consumers would fail, so **the run is FAILED and nothing is published**.
- **NON-BREAKING**: safe, so the run continues and records are checked one by one.

Change type	Classification	Meaning	Example
`MISSING_REQUIRED_COLUMN`	BREAKING	A required column is not in the file	currency is missing
`PRIMARY_KEY_COLUMN_MISSING`	BREAKING	The key column is not in the file	order_id is missing
`RENAME_CANDIDATE`	BREAKING (required column)	A column seems renamed without a mapping	unit_price → unit_price_amount
`TYPE_MISMATCH`	BREAKING	More than 50% of a column's values have the wrong type	quantity is mostly text like 'two'
`NULLABILITY_VIOLATION`	BREAKING	A non-nullable column is empty in every row	full_name all empty
`MISSING_OPTIONAL_COLUMN`	NON-BREAKING	An optional column is absent; NULL or the default is used	channel missing
`UNEXPECTED_COLUMN`	NON-BREAKING (BREAKING in STRICT)	Extra column not in the contract; it is ignored	gift_wrap added
`COLUMN_RENAME_MAPPED`	NON-BREAKING	An old column name accepted via the renamed_from mapping	email → email_address
`NULL_VALUES_IN_NON_NULLABLE`	NON-BREAKING	Only some rows are empty; those rows are rejected	a few product_id blank
`ALLOWED_VALUES_DRIFT`	NON-BREAKING (BREAKING in STRICT)	Some values are outside the allowed list; those rows are rejected	status = 'LOST'

Why '50%' for TYPE_MISMATCH?

One bad value is a **record** problem (only that row is rejected). When most of a column is wrong, the producer has changed the column itself, which is a **schema** problem. This avoids blocking a whole 100,000-row file because of one typo, while still catching a real format change.

BACKWARD vs STRICT policy

Each contract has a compatibility policy. **BACKWARD** (the default) allows safe additions such as extra columns. **STRICT** is for consumers that need an exact schema: unexpected columns and unknown values block the batch.

6. Level 4: Record errors (13 error codes)

These appear on the **Rejected Records** page. Only the bad rows are rejected; the good rows in the same batch are still published. **One row can have several errors at once**, and each is stored with a human-readable reason and the offending value.

Error code	Meaning	Example (orders)
`MALFORMED_ROW`	The row cannot be read	3 values in a row that should have 12
`NULL_NOT_ALLOWED`	A required value is empty	product_id is blank
`INVALID_TYPE`	Wrong data type	quantity = "two" or "2.5"
`INVALID_TIMESTAMP`	Not a real date or time	order_ts = "2024-02-30T25:61:00"
`INVALID_ALLOWED_VALUE`	Value not in the allowed list	status = "LOST"
`VALUE_BELOW_MIN`	Number below the minimum	quantity = -3
`VALUE_ABOVE_MAX`	Number above the maximum	quantity = 5000 (max 1000)
`VALUE_TOO_LONG`	Text longer than allowed	a name longer than 100 characters
`PATTERN_MISMATCH`	Wrong format	email = "not-an-email"
`DUPLICATE_PRIMARY_KEY`	Same key twice in the batch (the first is kept)	ORD-000017 appears twice
`DUPLICATE_BUSINESS_EVENT`	Same customer + product + time twice	the same purchase recorded twice
`FOREIGN_KEY_VIOLATION`	Referenced record does not exist	customer_id = "CUST-99999"
`BUSINESS_RULE_VIOLATION`	A business rule failed	total ≠ quantity × unit_price; ship_ts before order_ts
`FUTURE_TIMESTAMP`	Date is in the future	order_ts = 2200-01-01

Business rules in the demo

Dataset	Rule	Error code if broken
orders	total_amount = quantity × unit_price (± 0.01)	BUSINESS_RULE_VIOLATION
orders	ship_ts ≥ order_ts (cannot ship before ordering)	BUSINESS_RULE_VIOLATION
orders	updated_ts ≥ order_ts	BUSINESS_RULE_VIOLATION
orders	order_ts not in the future (5 min tolerance)	FUTURE_TIMESTAMP
orders	customer_id must exist in published customers	FOREIGN_KEY_VIOLATION
customers	updated_ts ≥ signup_ts	BUSINESS_RULE_VIOLATION
customers	signup_ts not in the future	FUTURE_TIMESTAMP

Example: one row with several errors

A row with `quantity = -1`, `currency = XYZ` and `customer_id = CUST-55555` gets **four** errors at once:

```
VALUE_BELOW_MIN (quantity) · INVALID_ALLOWED_VALUE (currency) · FOREIGN_KEY_VIOLATION
(customer_id) · BUSINESS_RULE_VIOLATION (total does not match)
```

The original row is stored exactly as received (quantity is still "-1"); nothing is silently changed or dropped.

Your demo file `orders.csv` contains **one example of each** of the main record errors (10 bad rows). The generator's `manifest.json` lists which row has which defect, and the tests prove the platform rejects exactly those rows.

7. Contract-version changes

These are checked when you **edit a contract** (Contracts page → New version), not when a file is uploaded. A breaking version can only be saved if you tick 'I acknowledge breaking changes'; older versions are never modified.

Change	BACKWARD policy	STRICT policy
Add an optional / nullable / defaulted column	NON-BREAKING	NON-BREAKING
Add a required column without a default	BREAKING	BREAKING
Remove a required column	BREAKING	BREAKING
Remove an optional column	NON-BREAKING	BREAKING
Rename with renamed_from mapping	NON-BREAKING	NON-BREAKING
Rename without mapping (required column)	BREAKING	BREAKING
Widen a type (integer → decimal, date → timestamp)	NON-BREAKING	NON-BREAKING
Any other type change	BREAKING	BREAKING
Nullable → non-nullable	BREAKING	BREAKING
Non-nullable → nullable	NON-BREAKING	BREAKING
Optional → required	BREAKING	BREAKING
Primary key changed	BREAKING	BREAKING
Allowed value added	NON-BREAKING	BREAKING
Allowed value removed	BREAKING	BREAKING
Range / max length tightened	BREAKING	BREAKING
Range / max length relaxed	NON-BREAKING	NON-BREAKING
Regex pattern changed	BREAKING	BREAKING

8. Where to see errors in the dashboard

Page	What it shows about errors
Data Ingestion	Level 1 errors (red message) and the final status of the batch you just uploaded
Pipeline Runs	Every batch with its status and failure reason; Retry button for FAILED runs
Validation Results	Schema validation report (Level 3), error counts per code and column, sample rejected rows
Rejected Records	Every rejected row; filter by run or error code; click for all reasons and the original row; Export CSV
Dashboard	Totals: rejected records, breaking/non-breaking changes, failed runs, top rejection reasons
AI Assistant	Plain-language explanation of the errors and suggested fixes

9. Quick summary and tips

Level	What fails	Result
1. Upload refused	The file	Error shown immediately; nothing stored

Level	What fails	Result
2. Run FAILED	The whole batch	Nothing published; reason saved; safe to retry
3. Schema change	The structure	BREAKING blocks the batch; NON-BREAKING continues
4. Record error	Single rows	Bad rows quarantined; good rows published

Tips to remember

- **Batch = one file.** One file creates one pipeline run.
- **Input = Valid + Rejected** (except when the whole batch is blocked or skipped).
- A **breaking** schema change stops the **whole batch**; a **record** error only stops **that row**.
- Nothing is ever silently dropped: every rejected row keeps its original values, row number, source file and reasons.
- Sending the same file twice never creates duplicates (SKIPPED_DUPLICATE, or 'unchanged' with force).
- The AI only explains errors; the Python rules decide pass or fail.